

CS324 Deep Learning Assignment 3 Report

Student: 12310520 Rui Yuhan

December 23, 2025

1 Part I: PyTorch LSTM

In the first part of Assignment 3, I implement a Long Short-Term Memory (LSTM) network from scratch using PyTorch and apply it to the palindrome prediction task.

1.1 Task 1: Implementation

I implement the LSTM architecture manually in `lstm.py` without relying on PyTorch's built-in `nn.LSTM` module. The implementation follows the skeleton provided in the assignment files.

Forward Propagation. The forward pass processes the input sequence $\mathbf{x} \in \mathbb{R}^{B \times T \times D}$ step-by-step. For each time step t from 0 to $T - 1$:

1. Extract the input at current step $\mathbf{x}^{(t)}$.
2. Compute the gates (input modulation g , input i , forget f , output o) using the current input $\mathbf{x}^{(t)}$ and previous hidden state $\mathbf{h}^{(t-1)}$.
3. Update the cell state $\mathbf{c}^{(t)}$ by combining the previous cell state $\mathbf{c}^{(t-1)}$ (modulated by the forget gate) and the new information (modulated by the input gate).
4. Update the hidden state $\mathbf{h}^{(t)}$ by applying $\tanh$ to the cell state and modulating it with the output gate.

The recurrence equations applied at each step are:

$$\begin{aligned}g^{(t)} &= \tanh(\mathbf{W}_{gx}\mathbf{x}^{(t)} + \mathbf{W}_{gh}\mathbf{h}^{(t-1)} + \mathbf{b}_g) \\i^{(t)} &= \sigma(\mathbf{W}_{ix}\mathbf{x}^{(t)} + \mathbf{W}_{ih}\mathbf{h}^{(t-1)} + \mathbf{b}_i) \\f^{(t)} &= \sigma(\mathbf{W}_{fx}\mathbf{x}^{(t)} + \mathbf{W}_{fh}\mathbf{h}^{(t-1)} + \mathbf{b}_f) \\o^{(t)} &= \sigma(\mathbf{W}_{ox}\mathbf{x}^{(t)} + \mathbf{W}_{oh}\mathbf{h}^{(t-1)} + \mathbf{b}_o) \\c^{(t)} &= g^{(t)} \odot i^{(t)} + c^{(t-1)} \odot f^{(t)} \\h^{(t)} &= \tanh(c^{(t)}) \odot o^{(t)}\end{aligned}$$

After processing the entire sequence, the final hidden state $\mathbf{h}^{(T-1)}$ is passed through a linear layer to produce the output logits:

$$p^{(T)} = \mathbf{W}_{ph}h^{(T-1)} + \mathbf{b}_p$$

The predicted probability distribution is given by the softmax function:

$$\tilde{y}^{(T)} = \text{softmax}(p^{(T)})$$

The network is trained to minimize the Cross-Entropy loss over the last time step:

$$\mathcal{L} = - \sum_{k=1}^K y_k \log(\tilde{y}_k^{(T)})$$

where y_k is the one-hot encoding of the target digit. In the PyTorch implementation, the `CrossEntropyLoss` combines the softmax and log-likelihood computation for numerical stability.

1.2 Task 2: Palindrome Prediction and RNN Comparison

In this task, the LSTM is trained to predict the T -th digit of a palindrome of length T , given the preceding $T - 1$ digits.

Experimental Setup. The model is trained and evaluated with the following configuration (based on the provided Jupyter notebook):

- **Dataset Size:** 10,000 samples (reduced from default 100,000 for faster iteration).
- **Palindrome Lengths (T):** Evaluated on lengths [5, 10, 15, 20, 25, 30].
- **Network Dimensions:** Input dimension 1, hidden dimension 128, output dimension 10.
- **Optimization:** RMSprop optimizer with learning rate 0.001.
- **Training:** 50 epochs, batch size 128, gradient clipping with max norm 10.0.

Results and Comparison. I trained the LSTM for each sequence length and compared its performance with the vanilla RNN from Assignment 2.

For a short sequence length ($T = 5$), the LSTM achieves perfect accuracy (Train Acc=1.0000, Val Acc=1.0000). More importantly, the LSTM maintains this perfect performance across all tested lengths up to $T = 30$.

Figure 1 shows the per-epoch loss and accuracy trajectories for the $T = 5$ experiment. Both training and validation losses decrease smoothly and their accuracies rapidly approach unity, confirming that the model converges reliably even with a modest dataset size. The curves are lightly down-sampled (every two epochs) to keep the visualization clear.

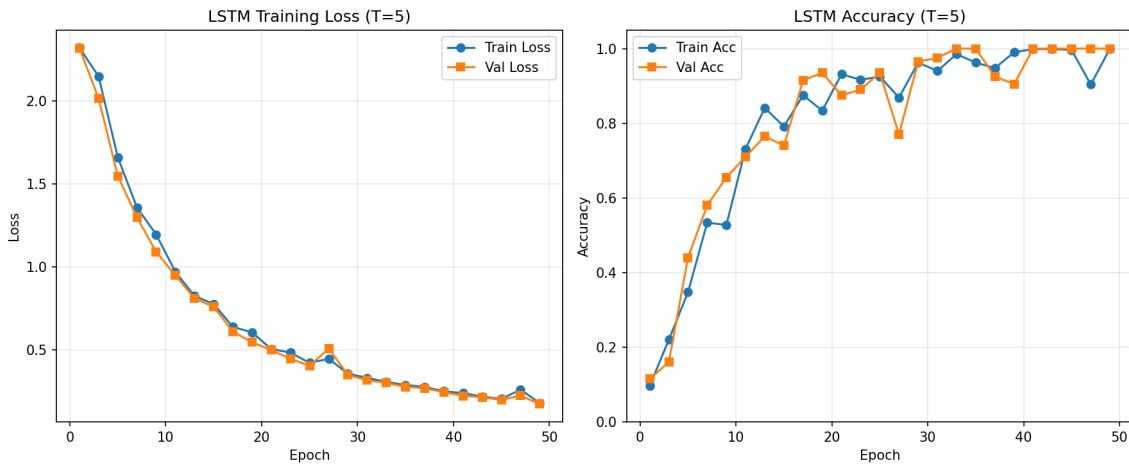


Figure 1: Training/validation loss and accuracy for $T = 5$. Down-sampled points illustrate the stable convergence towards perfect accuracy.

As shown in Figure 2, the LSTM significantly outperforms the vanilla RNN, especially as the sequence length increases. The vanilla RNN’s accuracy drops drastically for $T \geq 20$ due to the vanishing gradient problem. The LSTM, equipped with gating mechanisms, effectively mitigates this issue and successfully learns long-term dependencies.

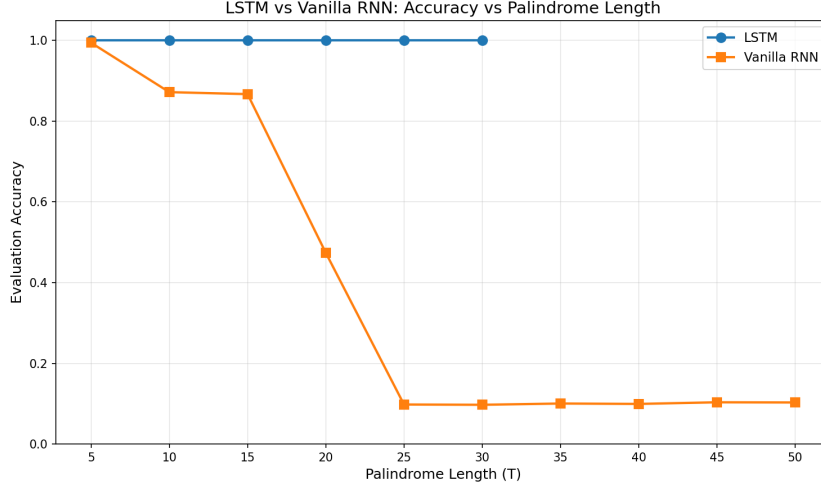


Figure 2: Comparison of training accuracy between LSTM and Vanilla RNN on the Palindrome dataset.

The vanilla RNN struggles to learn the dependency in the palindrome sequence due to the vanishing gradient problem, where gradients decay exponentially over time steps. In contrast, the LSTM’s gating mechanism (specifically the forget and input gates) allows it to maintain the cell state over longer intervals, effectively preserving the gradient information required to solve the task. This confirms the theoretical advantage of LSTMs in handling long-term dependencies.

2 Part II: Generative Adversarial Networks (GANs)

In this part, I implement a Generative Adversarial Network (GAN) to generate MNIST digits. The GAN consists of two networks, a Generator (G) and a Discriminator (D), trained in a min-max game.

2.1 Task 1: Implementation Details

Architecture. The model architectures are implemented as follows:

- **Generator (G):** Takes a latent vector $z \sim \mathcal{N}(0, I)$ of dimension 100 as input. It transforms the noise into a 28×28 image through a series of fully connected layers.
 - **Block 1:** Linear ($100 \rightarrow 128$) $\rightarrow$ LeakyReLU(0.2)
 - **Block 2:** Linear ($128 \rightarrow 256$) $\rightarrow$ BatchNorm1d $\rightarrow$ LeakyReLU(0.2)
 - **Block 3:** Linear ($256 \rightarrow 512$) $\rightarrow$ BatchNorm1d $\rightarrow$ LeakyReLU(0.2)
 - **Block 4:** Linear ($512 \rightarrow 1024$) $\rightarrow$ BatchNorm1d $\rightarrow$ LeakyReLU(0.2)
 - **Output:** Linear ($1024 \rightarrow 784$) $\rightarrow$ Tanh $\rightarrow$ Reshape to (1, 28, 28)
- **Discriminator (D):** Takes a flattened image (784) and outputs a probability of validity.
 - **Layer 1:** Linear ($784 \rightarrow 512$) $\rightarrow$ LeakyReLU(0.2)

- **Layer 2:** Linear (512 $\rightarrow$ 256) $\rightarrow$ LeakyReLU(0.2)
- **Output:** Linear (256 $\rightarrow$ 1) $\rightarrow$ Sigmoid

Forward Propagation. In the forward pass, the Generator G maps the latent vector z to the data space (images), while the Discriminator D maps an image (real or generated) to a scalar probability.

For the Generator:

$$\begin{aligned} h_1 &= \text{LReLU}(\mathbf{W}_{g1}z + \mathbf{b}_{g1}) \\ h_2 &= \text{LReLU}(\text{BN}(\mathbf{W}_{g2}h_1 + \mathbf{b}_{g2})) \\ h_3 &= \text{LReLU}(\text{BN}(\mathbf{W}_{g3}h_2 + \mathbf{b}_{g3})) \\ h_4 &= \text{LReLU}(\text{BN}(\mathbf{W}_{g4}h_3 + \mathbf{b}_{g4})) \\ \text{img} &= \text{Tanh}(\mathbf{W}_{out}h_4 + \mathbf{b}_{out}) \end{aligned}$$

For the Discriminator taking an input x (flattened image):

$$\begin{aligned} d_1 &= \text{LReLU}(\mathbf{W}_{d1}x + \mathbf{b}_{d1}) \\ d_2 &= \text{LReLU}(\mathbf{W}_{d2}d_1 + \mathbf{b}_{d2}) \\ \text{validity} &= \text{Sigmoid}(\mathbf{W}_{d3}d_2 + \mathbf{b}_{d3}) \end{aligned}$$

Training Objective. The network is trained to optimize the standard GAN min-max objective:

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{data}} [\log D(x)] + \mathbb{E}_{z \sim p_z} [\log(1 - D(G(z)))]$$

Implementation Details: In practice, I implement this objective using the Binary Cross Entropy (BCE) loss:

$$\mathcal{L}_{BCE}(y, \hat{y}) = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

The training proceeds in two steps for each batch:

1. **Train Discriminator:** Maximize the probability of correctly classifying real and fake images.

$$\mathcal{L}_D = \mathcal{L}_{BCE}(1, D(x)) + \mathcal{L}_{BCE}(0, D(G(z)))$$

2. **Train Generator:** Minimize $\log(1 - D(G(z)))$. In practice, we maximize $\log(D(G(z)))$ for better gradients early in training.

$$\mathcal{L}_G = \mathcal{L}_{BCE}(1, D(G(z)))$$

I use the Adam optimizer with learning rate 0.0002 and betas (0.5, 0.999) for both networks. The training runs for 100 epochs.

Figure 3 summarizes how the discriminator and generator losses evolve throughout training. The discriminator initially learns faster, causing the generator loss to spike, but the two curves gradually stabilize and oscillate within a reasonable band, indicating that the adversarial game has reached a rough equilibrium. The smoothing overlay highlights the long-term trends beyond the noisy per-iteration fluctuations.

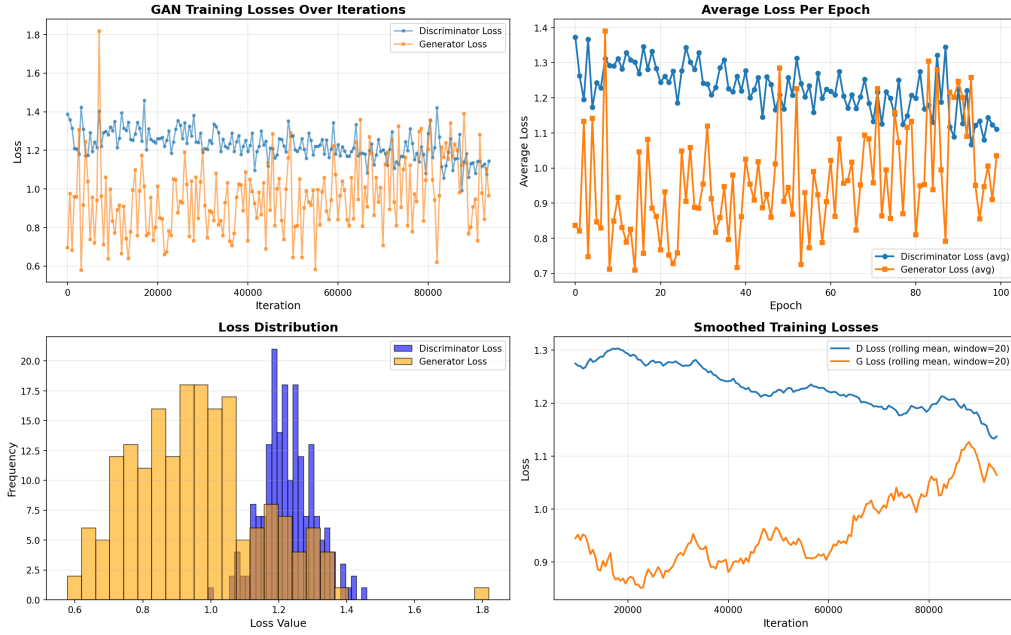


Figure 3: Training loss trajectory of the GAN. The solid lines show raw measurements while the smoother curves reveal the overall trend as training converges.

Complementing the loss curves, Figure 4 tracks the discriminator and generator accuracies. The discriminator settles around 60–70% on both real and fake inputs while maintaining a modest gap, which is typical for a balanced GAN where neither player dominates completely. The generator’s fooling rate plateaus near 30%, reflecting that it consistently produces samples that are occasionally convincing but still leaves room for improvement (e.g., by increasing model capacity or training time).

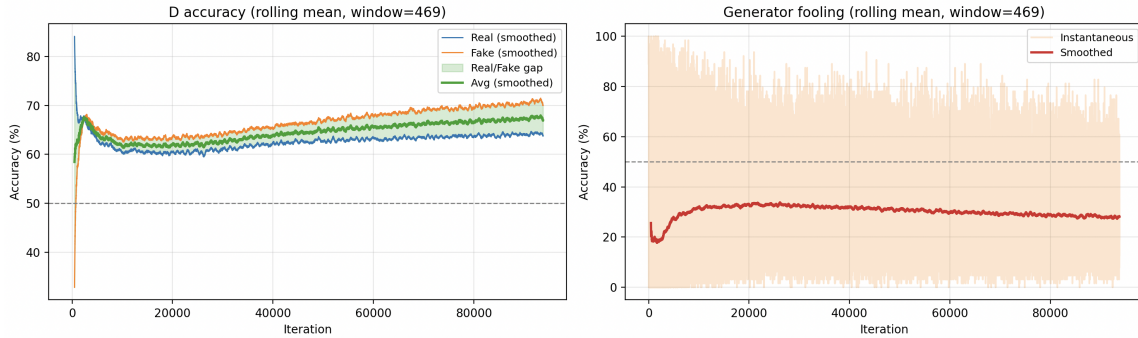


Figure 4: Rolling-mean accuracy curves. Left: discriminator real/fake accuracies and their gap. Right: generator bluffing rate with instantaneous and smoothed trends.

2.2 Task 2: Generated Samples

Figure 5 shows the progression of generated images during training.

- **Start (Iteration 0):** The generator is initialized with random weights, resulting in unstructured noise that carries no semantic meaning. The discriminator easily distinguishes these from real images.
- **Halfway:** The model begins to capture the structural properties of the dataset. We observe the emergence of strokes and rough digit shapes, though some artifacts and blur-

riness remain. This transition indicates that the generator is starting to learn the data manifold under the guidance of the discriminator.

- **End:** By the end of the training (100 epochs), the generated samples are sharp and clearly recognizable as handwritten digits. The generator successfully mimics the distribution of the MNIST dataset. Furthermore, the diversity in the generated batch suggests that the model has avoided severe mode collapse, a common failure mode in GANs where the generator produces only a limited set of outputs.

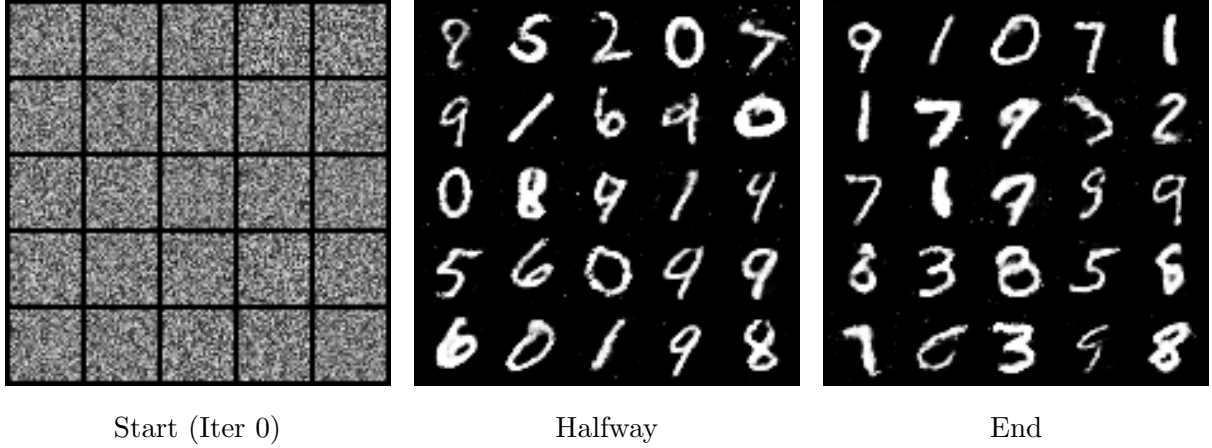


Figure 5: Generated samples at the start, middle, and end of training.

2.3 Task 3: Latent Space Interpolation

To verify that the GAN has learned a meaningful and continuous latent space representation rather than simply memorizing the training data, I perform linear interpolation between two random latent vectors z_1 and z_2 .

The interpolated latent vector z' is computed as:

$$z' = (1 - \alpha)z_1 + \alpha z_2$$

where α varies from 0 to 1 in 7 evenly spaced intermediate steps (total 9 images).

Figure 6 displays the transition between two distinct digit classes. As we move from left to right, we can observe a smooth semantic morphing:

- The starting digit (leftmost) gradually transforms.
- Intermediate shapes appear that possess characteristics of both digits, showing how the generator navigates the decision boundary between classes.
- The transformation is continuous, with strokes fading in or shifting position fluidly, rather than abrupt jumps.

This smooth transformation confirms that the generator has learned the underlying manifold of the handwritten digits, allowing it to generalize to unseen data points within the latent space.

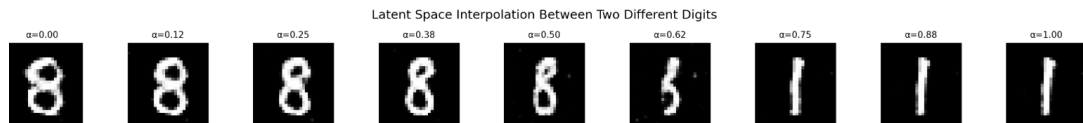


Figure 6: Interpolation between two digits in the latent space. The smooth transition indicates that the generator has learned the manifold of handwritten digits.